Taller 1: Teoría de Aprendizaje de Máquina – 2026-2

Profesor: Diego Armando Pérez Rosero

Tema: Clasificación supervisada, vecinos cercanos y evaluación de modelos

Instrucciones

- Responda los tres puntos de manera clara y justifique los procedimientos principales.
- Presente los cálculos manuales indicados y el código en Python con resultados, figuras y una breve discusión.
- Use una semilla fija en los experimentos y explique el significado de las métricas reportadas.
- Se permite el uso de `NumPy`, `Matplotlib` y `scikit-learn`, salvo cuando se solicite implementar una regla manualmente.

Preguntas

1. **Clasificación lineal y algoritmo del perceptrón.**
Considere un clasificador binario para observaciones $x \in \mathbb{R}^2$, con etiquetas $y \in \{-1, +1\}$, cuya predicción se obtiene mediante

$$\hat{y} = \begin{cases} +1, & w^\top x + b > 0, \\ -1, & w^\top x + b \leq 0. \end{cases} \qquad w \in \mathbb{R}^2, \quad b \in \mathbb{R}.$$

1.1 **Frontera de decisión.** Sea $w = [1, 2]^\top$ y $b = -3$. Obtenga la ecuación de la recta $w^\top x + b = 0$, dibújela e indique la clase asignada a $x_1 = (0, 0)$, $x_2 = (1, 0)$, $x_3 = (2, 1)$ y $x_4 = (3, 1)$. Justifique cada predicción.

1.2 **Actualización del perceptrón.** Cuando una observación (x_i, y_i) está mal clasificada, considere la regla

$$\boxed{w_{\text{nuevo}} = w + \eta y_i x_i, \qquad b_{\text{nuevo}} = b + \eta y_i, \quad \eta > 0.}$$

Defina el margen de la observación como $m_i = y_i(w^\top x_i + b)$ y **demuestre** que, inmediatamente después de aplicar la actualización sobre esa misma observación,

$$\boxed{m_{i,\text{nuevo}} = m_i + \eta(\|x_i\|_2^2 + 1).}$$

Explique brevemente qué implica este resultado para la observación corregida.

1.3 **Ejercicio numérico.** Con $w = [0, 0]^\top$, $b = 0$, $\eta = 0{,}5$, $x = [2, 1]^\top$ y $y = +1$, determine si corresponde actualizar los parámetros usando la regla de predicción dada. De ser así, calcule los nuevos w, b y margen.

2. **Clasificación por los k vecinos más cercanos (k-NN).**
Considere una observación nueva $q = (1, 1)$ y el siguiente conjunto de entrenamiento:

Observación	x_1	x_2	x_3	x_4	x_5
x_i	$(0,0)$	$(1,0)$	$(3,1)$	$(0,3)$	$(1{,}1, 1{,}1)$
Clase	A	A	B	B	B

Use la distancia euclidiana $d(q, x_i) = \sqrt{(q_1 - x_{i,1})^2 + (q_2 - x_{i,2})^2}$ y la votación mayoritaria de los k vecinos más próximos.

2.1 **Cálculo manual.** Calcule las cinco distancias y ordene las observaciones desde la más cercana hasta la más lejana. Clasifique q utilizando $k = 1$, $k = 3$ y $k = 5$. Explique por qué el valor de k puede cambiar la predicción.

2.2 **Implementación.** Genere $N = 300$ observaciones bidimensionales con la función `make_moons`, usando `noise=0.20` y `random_state=42`. Separe 70 % para entrenamiento y 30 % para prueba, estratificando las clases. Entrene `KNeighborsClassifier` con $k \in \{1, 5, 15\}$. Reporte la exactitud (*accuracy*)

en entrenamiento y prueba para cada caso. Dibuje los datos y, al menos para $k = 1$ y $k = 15$, sus regiones de decisión.

2.3 **Interpretación.** Discuta qué ocurre al escoger un k muy pequeño o muy grande. Indique cuál de los modelos generalizó mejor en esta evaluación y justifique su elección. Explique por qué la selección formal del mejor k debe realizarse con validación, no con los datos de prueba.

3. **Evaluación de un clasificador binario.**
Sea un conjunto de diez etiquetas verdaderas y y sus predicciones $\hat{y}$:

$$y = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1], \qquad \hat{y} = [0, 1, 0, 0, 1, 0, 1, 1, 0, 1].$$

Tome la clase 1 como positiva. Recuerde que TP y TN representan aciertos positivos y negativos, mientras que FP y FN son los errores correspondientes.

3.1 **Matriz de confusión.** Construya la matriz de confusión con filas correspondientes a clases reales y columnas a clases predichas. Identifique TP, TN, FP y FN.

3.2 **Métricas.** Calcule manualmente las siguientes medidas y explique qué evalúa cada una:

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}}$$

$$\boxed{\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}}.}$$

3.3 **Verificación y análisis.** En Python, compruebe sus resultados usando las funciones `confusion_matrix` y `accuracy_score`, además de `precision_score`, `recall_score` y `f1_score`. Escriba dos situaciones en las que la exactitud por sí sola podría resultar engañosa.

Entrega: un documento PDF con procedimientos, figuras y conclusiones, acompañado de un cuaderno `.ipynb` reproducible.